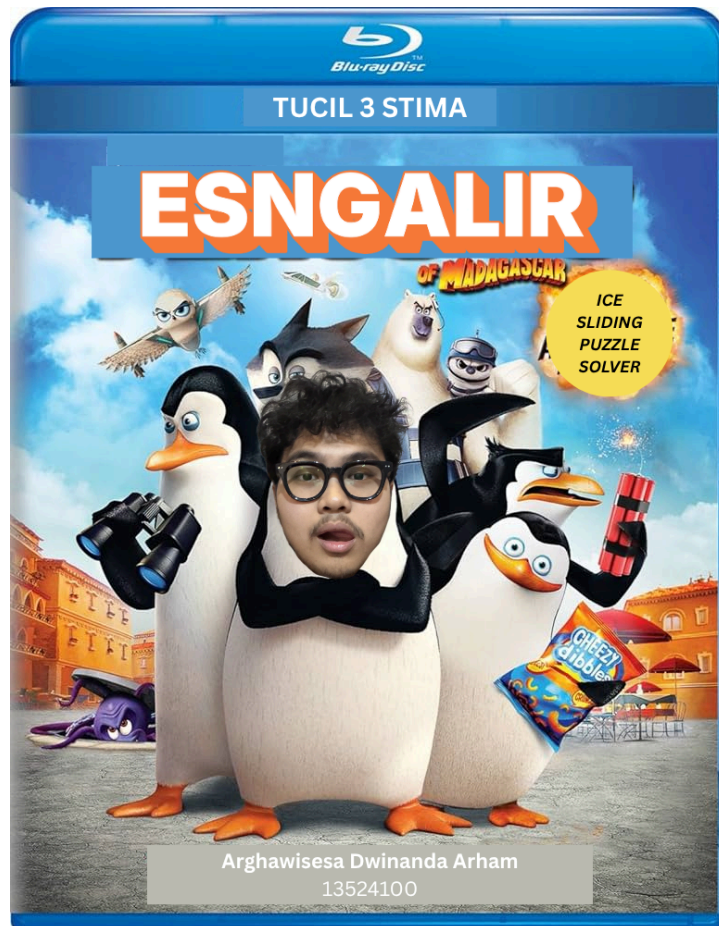


LAPORAN TUGAS KECIL 3

IF2211: Strategi Algoritma



EsNgalir: Ice Sliding Puzzle Solver

Arghawisesa Dwinanda Arham

13524100

K02

Semester II Tahun 2025/2026
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

Daftar Isi.....	1
Algoritma Pathfinding.....	3
Uniform Cost Search.....	3
Greedy Best First Search (GBFS).....	3
A* (A-Star).....	4
BFS (Breadth First Search) (BONUS).....	5
DFS (Depth First Search) (BONUS).....	5
Analisis Algoritma Pathfinding.....	6
Definisi $f(n)$ dan $g(n)$	6
Admissibilitas Heuristik pada A*.....	6
Perbandingan Teoritis Algoritma.....	7
Implementasi.....	9
Bahasa dan Struktur.....	9
Flowchart Alur Program.....	10
Penjelasan Komponen.....	12
Model.....	12
Solver.....	13
Heuristic.....	14
IO.....	15
UI.....	15
Bonus.....	15
Algoritma Tambahan 1 (BFS).....	15
Algoritma Tambahan 2 (DFS).....	16
Heuristik Bonus 1 (CheckpointAware).....	16
Heuristik Bonus 2 (Euclidean).....	17
GUI.....	18
Uji Coba.....	20
Test Case 1.....	20
Test Case 2.....	22
Test Case 3.....	24
Test Case 4.....	27
Test Case 5.....	29
Analisis dan Kesimpulan.....	33
Analisis.....	33
Rekapitulasi Hasil Uji Coba.....	33
Analisis per Algoritma.....	34
Analisis per Algoritma.....	35
Kesimpulan.....	36
Lampiran.....	38

Referensi.....	39
----------------	----

Algoritma Pathfinding

Uniform Cost Search

UCS adalah algoritma pencarian yang mengeksplorasi node berdasarkan biaya kumulatif dari titik awal ke node tersebut. UCS menjamin solusi optimal selama semua biaya bernilai non-negatif.

Langkah-langkah UCS pada EsNgalir adalah sebagai berikut:

1. Inisialisasi priority queue (min-heap) dengan state awal berisi posisi karakter Z, nextCheckpoint=0, gCost=0.
2. Inisialisasi visited set (HashSet) yang kosong.
3. Selama priority queue tidak kosong:
 - a. Ambil state dengan gCost terkecil dari priority queue.
 - b. Jika state sudah ada di visited, lewati (skip).
 - c. Tambahkan state ke visited.
 - d. Jika state adalah goal (berhenti di O dan semua checkpoint sudah dilalui), rekonstruksi jalur dan kembalikan solusi.
 - e. Untuk setiap arah (UP, DOWN, LEFT, RIGHT):
 - Simulasikan gerakan sliding dari posisi sekarang.
 - Jika gerakan valid (tidak game over, tidak noMove), hitung gCost baru = gCost sekarang + moveCost, buat state baru dengan posisi akhir sliding, checkpoint yang sudah diperbarui, dan gCost baru, lalu jika state baru belum di visited masukkan ke priority queue.
4. Jika priority queue kosong dan goal belum ditemukan, kembalikan "tidak ada solusi".

Pada UCS, nilai $f(n) = g(n)$, sehingga priority queue mengurutkan state semata-mata berdasarkan biaya nyata dari titik awal.

Greedy Best First Search (GBFS)

GBFS adalah algoritma pencarian informed yang mengeksplorasi node berdasarkan estimasi biaya ke goal (heuristik) tanpa mempertimbangkan biaya yang sudah dikeluarkan.

Langkah-langkah GBFS pada EsNgalir:

1. Inisialisasi priority queue dengan state awal, di mana prioritas ditentukan oleh $h(\text{state}) = \text{estimasi ke goal}$.

2. Inisialisasi visited set yang kosong.
3. Selama priority queue tidak kosong:
 - a. Ambil state dengan $h(n)$ terkecil.
 - b. Jika sudah di visited, lewati.
 - c. Tambahkan ke visited.
 - d. Jika goal, rekonstruksi dan kembalikan solusi.
 - e. Untuk setiap arah, simulasikan sliding, jika valid hitung $h(\text{next})$ menggunakan fungsi heuristik yang dipilih, buat state baru dengan $f\text{Cost} = h(\text{next})$, lalu jika belum di visited masukkan ke priority queue.
4. Jika tidak ditemukan, kembalikan "tidak ada solusi".

GBFS tidak menjamin solusi optimal karena ia hanya mempertimbangkan estimasi ke goal dan mengabaikan biaya yang sudah dikeluarkan.

A* (A-Star)

A* menggabungkan keunggulan UCS (mempertimbangkan biaya nyata) dan GBFS (mempertimbangkan estimasi ke goal). A* menjamin solusi optimal jika heuristik yang digunakan bersifat admissible.

Langkah-langkah A* pada EsNgalir:

1. Inisialisasi priority queue dengan state awal, di mana prioritas ditentukan oleh $f(n) = g(n) + h(n)$.
2. Inisialisasi visited set yang kosong.
3. Selama priority queue tidak kosong:
 - a. Ambil state dengan $f(n)$ terkecil.
 - b. Jika sudah di visited, lewati.
 - c. Tambahkan ke visited.
 - d. Jika goal, rekonstruksi dan kembalikan solusi.
 - e. Untuk setiap arah, simulasikan sliding, hitung $g\text{Cost}$ baru = $g\text{Cost} + \text{moveCost}$, hitung $h(\text{next})$ menggunakan fungsi heuristik, buat state baru dengan $f\text{Cost} = g\text{Cost} + h(\text{next})$, lalu jika belum di visited masukkan ke priority queue.
4. Jika tidak ditemukan, kembalikan "tidak ada solusi".

BFS (Breadth First Search) (BONUS)

BFS adalah algoritma pencarian uninformed yang mengeksplorasi semua node pada kedalaman (jumlah langkah) yang sama sebelum berpindah ke kedalaman berikutnya.

Langkah-langkah BFS pada EsNgalir:

1. Inisialisasi queue (FIFO) dengan state awal dan tambahkan state awal ke visited set.
2. Selama queue tidak kosong:
 - a. Ambil state dari depan queue dan tambahkan ke counter iterasi.
 - b. Jika goal, rekonstruksi dan kembalikan solusi.
 - c. Untuk setiap arah, simulasikan sliding, jika valid dan belum di visited tambahkan ke visited lalu masukkan ke belakang queue.
3. Jika tidak ditemukan, kembalikan "tidak ada solusi".

BFS menjamin solusi dengan jumlah gerakan paling sedikit, namun tidak menjamin solusi dengan cost paling kecil karena tidak mempertimbangkan bobot tiap tile.

DFS (Depth First Search) (BONUS)

DFS adalah algoritma pencarian uninformed yang mengeksplorasi satu jalur sedalam mungkin sebelum mundur (backtrack) dan mencoba jalur lain.

Langkah-langkah DFS pada EsNgalir:

1. Inisialisasi stack (LIFO) dengan state awal dan tambahkan state awal ke visited set.
2. Selama stack tidak kosong:
 - a. Ambil state dari atas stack dan tambahkan ke counter iterasi.
 - b. Jika goal, rekonstruksi dan kembalikan solusi.
 - c. Untuk setiap arah, simulasikan sliding, jika valid dan belum di visited tambahkan ke visited lalu push ke atas stack.
3. Jika tidak ditemukan, kembalikan "tidak ada solusi".

DFS tidak menjamin solusi optimal dan bisa menghasilkan jalur yang sangat panjang, namun DFS hemat memori karena hanya perlu menyimpan satu jalur pada satu waktu.

Analisis Algoritma Pathfinding

Definisi $f(n)$ dan $g(n)$

$g(n)$ pada semua algoritma didefinisikan sebagai total biaya (cost) yang sudah dikeluarkan dari state awal hingga state n . Pada Ice Sliding Puzzle, $g(n)$ dihitung sebagai jumlah cost semua tile yang dilalui selama seluruh gerakan sliding dari awal hingga mencapai state n , di mana tile posisi awal setiap gerakan tidak dihitung namun tile posisi akhir dihitung.

$f(n)$ pada masing-masing algoritma:

1. **UCS**: $f(n) = g(n)$. Hanya mempertimbangkan biaya nyata, tidak menggunakan heuristik.
2. **GBFS**: $f(n) = h(n)$. Hanya mempertimbangkan estimasi ke goal, tidak mempertimbangkan biaya yang sudah dikeluarkan.
3. **A***: $f(n) = g(n) + h(n)$. Mempertimbangkan biaya nyata dan estimasi ke goal, menghasilkan perkiraan total biaya solusi melalui n .
4. **BFS**: $f(n) = \text{kedalaman (jumlah langkah)}$. Semua node diperlakukan sama, tidak ada bobot.
5. **DFS**: Tidak ada fungsi evaluasi eksplisit. Urutan eksplorasi ditentukan oleh struktur stack.

Admissibilitas Heuristik pada A*

Sebuah heuristik $h(n)$ dikatakan admissible jika $h(n) \leq h^*(n)$ untuk semua n , di mana $h^*(n)$ adalah biaya sesungguhnya dari n ke goal. Heuristik tidak boleh overestimate biaya ke goal.

Heuristik pertama menggunakan **Manhattan Distance**:
 $h(n) = |row_n - row_{goal}| + |col_n - col_{goal}|$. Apakah Admissible? Ya. Karakter hanya bisa bergerak 4 arah (atas, bawah, kiri, kanan). Manhattan distance merepresentasikan jumlah tile minimum yang harus dilalui untuk mencapai goal tanpa hambatan apapun. Karena setiap tile memiliki cost minimal 0 dan Manhattan distance menghitung jumlah tile minimum, maka $h_{manhattan}(n) \leq \text{jumlah tile sesungguhnya} \leq \text{biaya sesungguhnya}$, sehingga Manhattan distance tidak pernah overestimate.

Heuristik kedua menggunakan **Checkpoint Aware**, di mana jika masih ada checkpoint yang belum dilalui, $h(n) = \text{manhattan}(n, cp_{next}) + \text{manhattan}(cp_{next}, goal)$. Jika semua checkpoint sudah dilalui, $h(n) = \text{manhattan}(n, goal)$. Apakah Admissible? Ya. Karakter wajib melewati checkpoint berikutnya sebelum mencapai goal, sehingga biaya minimum adalah biaya dari n ke checkpoint ditambah biaya dari checkpoint ke goal. Karena kita menggunakan Manhattan distance (yang sudah terbukti admissible) untuk mengestimasi masing-masing bagian, dan tidak ada jalan yang lebih pendek dari Manhattan distance, maka estimasi ini tidak pernah overestimate biaya sesungguhnya.

Heuristik ketiga menggunakan **Euclidean Distance**: $h(n) = \sqrt{(r_n - r_{goal})^2 + (c_n - c_{goal})^2}$. Apakah Admissible? Ya. Euclidean distance adalah jarak garis lurus antara dua titik. Pada grid dengan gerakan 4 arah, jarak sesungguhnya selalu $\geq$ Euclidean distance karena karakter tidak bisa bergerak diagonal, sehingga $h_{euclidean}(n) \leq h_{manhattan}(n) \leq h^*(n)$. Euclidean admissible namun kurang akurat dibanding Manhattan karena estimasinya lebih rendah.

Perbandingan Teoritis Algoritma

Algoritma	Complete?	Optimal?	Time	Space
UCS	Ya	Ya	$O(b^{C/\epsilon})$	$O(b^{C/\epsilon})$
GBFS	Tidak	Tidak	$O(b^m)$	$O(b^m)$
A*	Ya	Ya*	$O(b^m)$	$O(b^m)$
BFS	Ya	Tidak**	$O(b^d)$	$O(b^d)$
DFS	Tidak***	Ya*	$O(b^m)$	$O(b \cdot m)$

Keterangan: b = branching factor (maks 4 arah), d = kedalaman solusi (jumlah langkah), m = kedalaman maksimum search tree, C = biaya solusi optimal, e = biaya minimum tiap langkah, * = optimal jika heuristik admissible, ** = optimal dalam jumlah langkah bukan cost, *** = tidak complete jika ada cycle tanpa visited set.

UCS menjamin solusi optimal karena mengeksplorasi berdasarkan biaya nyata dan cocok sebagai baseline pembandingan, namun kelemahannya adalah lambat karena tidak menggunakan heuristik sehingga mengeksplorasi banyak node yang tidak relevan.

GBFS paling cepat menemukan solusi karena langsung menuju arah goal, namun solusinya sering tidak optimal karena mengabaikan biaya yang sudah dikeluarkan dan bisa terjebak di local minima jika papan kompleks.

A* merupakan keseimbangan terbaik antara kecepatan dan optimalitas. Dengan heuristik CheckpointAware yang akurat, A* dapat menemukan solusi optimal dengan iterasi lebih sedikit dari UCS, sebagaimana terlihat di hasil uji di mana UCS membutuhkan 14 iterasi sedangkan A* hanya 13 iterasi dengan solusi yang sama optimal.

BFS menjamin solusi dengan gerakan paling sedikit namun tidak menjamin solusi dengan cost paling kecil, sehingga tidak ideal untuk soal ini yang mengutamakan minimasi cost.

DFS tidak menjamin solusi optimal sama sekali dan berguna hanya untuk membuktikan keberadaan solusi. DFS bisa sangat lambat di papan besar karena mengeksplorasi jalur yang sangat dalam sebelum backtrack.

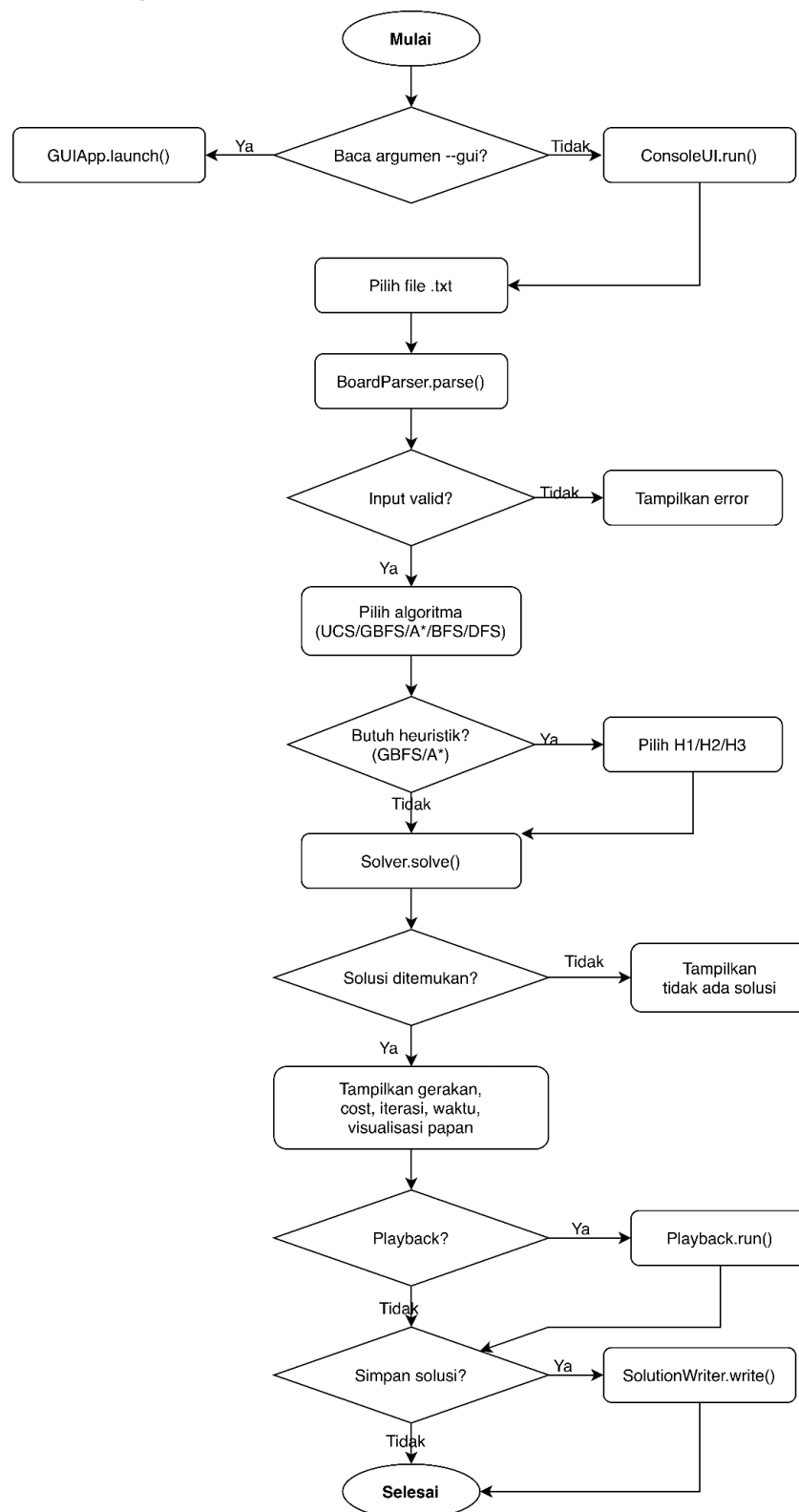
Implementasi

Bahasa dan Struktur

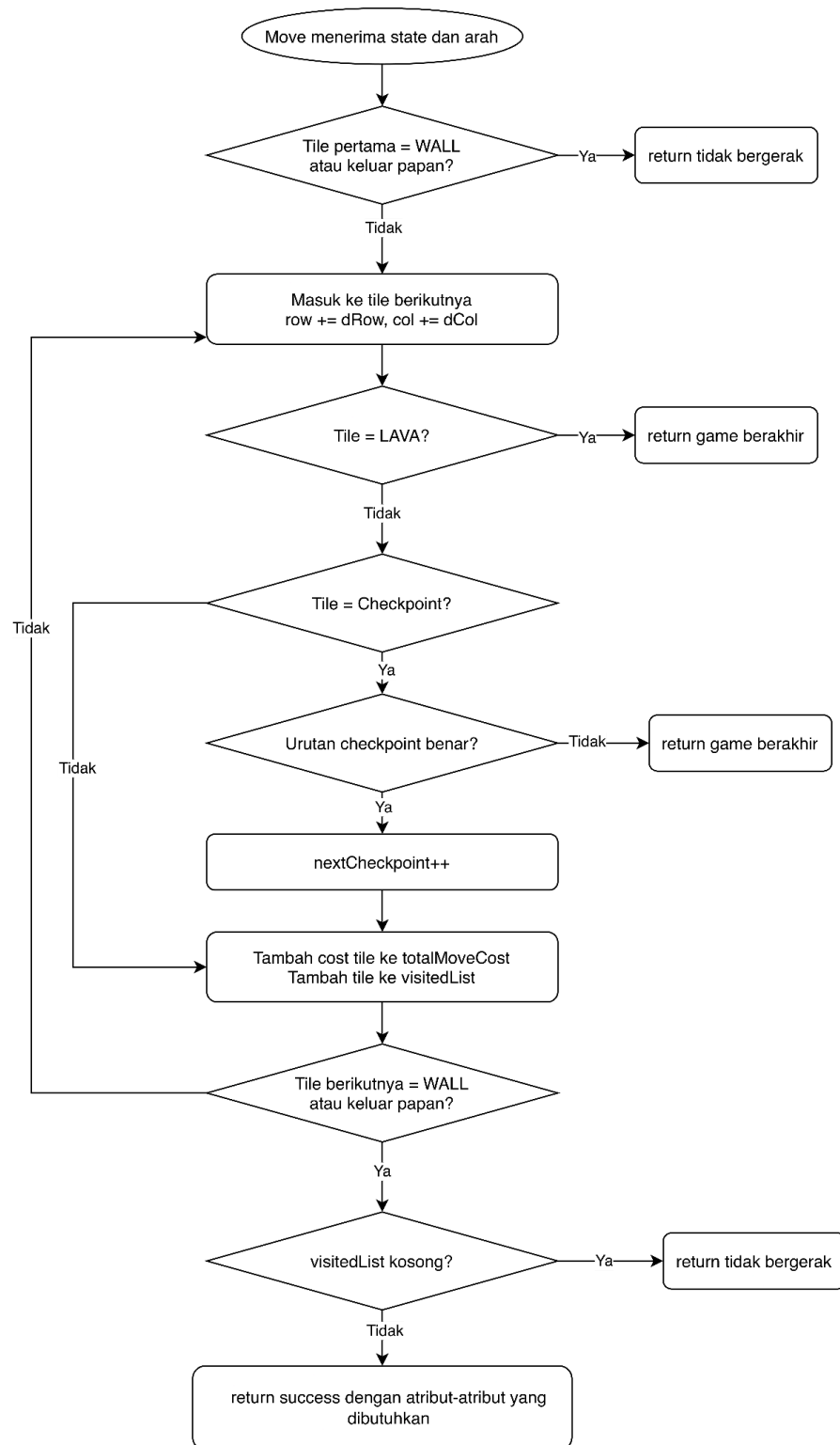
Program ditulis dalam bahasa Java dengan package utama "esngalir", menggunakan JavaFX untuk GUI. Dapat dikompilasi dan dijalankan pada Windows, Linux, dan macOS dengan Java 17+ dan JavaFX 17+. Struktur file yang digunakan adalah sebagai berikut:

```
src/esngalir/
├── Main.java           -> entry point
├── model/              -> representasi dunia permainan
│   ├── Direction.java -> enum 4 arah gerak
│   ├── Tile.java      -> enum tipe kotak papan
│   ├── Board.java     -> papan + logic sliding
│   ├── State.java     -> snapshot kondisi pion
│   ├── MoveResult.java -> hasil 1 gerakan sliding
│   └── SolverResult.java -> hasil keseluruhan pencarian
├── solver/             -> algoritma pencarian
│   ├── Solver.java    -> abstract class (template)
│   ├── UCSSolver.java -> UCS
│   ├── GBFSSolver.java -> GBFS
│   ├── AStarSolver.java -> A*
│   ├── BFSolver.java  -> BFS (bonus)
│   └── DFSolver.java  -> DFS (bonus)
├── heuristic/          -> fungsi heuristik
│   ├── Heuristic.java -> interface
│   ├── Manhattan.java -> H1 Manhattan Distance
│   ├── CheckpointAware.java -> H2 Checkpoint Aware (bonus)
│   └── Euclidean.java -> H3 Euclidean Distance (bonus)
├── io/                 -> baca/tulis file
│   ├── BoardParser.java -> parse .txt -> Board
│   └── SolutionWriter.java -> tulis solusi -> .txt
├── ui/                -> antarmuka pengguna
│   ├── ConsoleUI.java -> CLI
│   ├── Playback.java  -> playback CLI
│   ├── GUIApp.java    -> entry ke JavaFX (bonus)
│   ├── BoardPanel.java -> logic papan JavaFX (bonus)
│   └── ControlPanel.java -> logic playback GUI (bonus)
└── utils/
    ├── Timer.java     -> stopwatch eksekusi
    └── InputValidator.java -> validasi input
```

Flowchart Alur Program



Gambar 1. Flowchart Utama



Gambar 2. Flowchart Logika Gerak

Penjelasan Komponen

Model

Direction.java mendefinisikan 4 arah gerak sebagai enum dengan field deltaRow dan deltaCol yang langsung mengencode perubahan koordinat, sehingga tidak diperlukan if-else di tempat lain.

```
public enum Direction {  
    UP('U', -1, 0), DOWN('D', 1, 0),  
    LEFT('L', 0, -1), RIGHT('R', 0, 1);  
    ...  
}
```

Tile.java mendefinisikan semua tipe kotak sebagai enum. Method isCheckPoint() dan getCheckPointNumber() mengenkapsulasi logika checkpoint sehingga tidak tersebar di seluruh kode.

State.java merepresentasikan snapshot kondisi pion, seperti posisi (row, col), checkpoint berikutnya, gCost, fCost, parent, dan arah gerak. equals() dan hashCode() di-override berdasarkan (row, col, nextCheckPoint) sehingga HashSet dapat mendeteksi state yang sudah dikunjungi dengan benar.

```
@Override  
public boolean equals(Object o) {  
    if (!(o instanceof State)) return false;  
    State other = (State) o;  
    return row == other.getRow() && col == other.getCol()  
        && nextCheckPoint == other.getNextCheckPoint();  
}
```

Board.java adalah class terpenting yang menyimpan grid tile dan cost, serta mengimplementasikan fisika sliding di method move(). Logika move() mensimulasikan karakter meluncur ke satu arah sampai menabrak dinding, sambil menangani lava (gameOver), checkpoint (urutan benar/salah), dan keluar papan (gameOver).

```
public MoveResult move(State state, Direction dir) {  
    // cek tile pertama  
    if (!inGrid(nextRow, nextCol) || grid[nextRow][nextCol] ==  
    Tile.WALL)  
        return MoveResult.noMove();  
    // loop sliding
```

```

        while (true) {
            if (!inGrid(tempRow, tempCol)) return MoveResult.gameOver();
            if (grid[tempRow][tempCol] == Tile.WALL) break;
            if (tempTile == Tile.LAVA) return MoveResult.gameOver();
            if (tempTile.isCheckPoint()) { ... }
            totalMoveCost += cost[row][col];
        }
        return MoveResult.success(row, col, totalMoveCost, checkPoint,
visitedList);
    }

```

Solver

Solver.java adalah abstract class yang mengimplementasikan Template Method Pattern. Method solve() berisi alur pencarian yang sama untuk UCS, GBFS, dan A* menggunakan PriorityQueue. Method withFCost() adalah satu-satunya yang abstrak dan wajib diimplementasi subclass, berisi rumus fCost masing-masing algoritma.

```

public SolverResult solve() {
    PriorityQueue<State> openSet = new PriorityQueue<>(
        new Comparator<State>() {
            public int compare(State a, State b) {
                return Integer.compare(a.getFCost(), b.getFCost());
            }
        });
    Set<State> visited = new HashSet<>();
    // ... ambil state, cek visited, cek goal, ekspansi 4 arah
}

```

UCSSolver.java mengimplementasikan UCS dengan menetapkan fCost = gCost. Tidak ada heuristik -- priority queue mengurutkan semata-mata berdasarkan biaya nyata dari titik awal.

```

protected State withFCost(State state) {
    return new State(..., state.getGCost(), state.getGCost(), ...);
    // fCost == gCost karena gaada heuristic
}

```

GBFSSolver.java mengimplementasikan GBFS dengan menetapkan fCost = h(state). Biaya nyata (gCost) tetap disimpan di state untuk keperluan rekonstruksi path dan laporan cost, namun tidak digunakan sebagai penentu prioritas eksplorasi.

```

protected State withFCost(State state) {

```

```

    int h = heuristic.estimate(state, board);
    return new State(..., state.getGCost(), h, ...);
    // fCost = h saja, gCost diabaikan untuk prioritas
}

```

AStarSolver.java mengimplementasikan A* dengan menetapkan $fCost = gCost + h(state)$. Ini adalah kombinasi biaya nyata dan estimasi ke goal yang menjamin solusi optimal selama heuristik admissible.

```

protected State withFCost(State state) {
    int g = state.getGCost();
    int h = heuristic.estimate(state, board);
    return new State(..., g, g + h, ...);
}

```

BFSSolver.java dan DFSSolver.java tidak extend Solver karena menggunakan struktur data yang berbeda (Queue/Deque FIFO/LIFO bukan PriorityQueue). BFS menandai visited saat add ke queue, sedangkan solver berbasis PriorityQueue menandai visited saat poll karena satu state bisa masuk queue berkali-kali.

```

// BFS - Queue FIFO
Queue<State> queue = new ArrayDeque<>();

// DFS - Stack LIFO
Deque<State> stack = new ArrayDeque<>();
stack.push(next); // masuk ke atas
stack.pop();      // keluar dari atas

```

Heuristic

Heuristic adalah interface dengan satu method `estimate(State, Board)` yang mengembalikan estimasi biaya ke goal. Tiga heuristic yang disediakan yaitu Manhattan, CheckpointAware, dan Euclidean. Solver yang membutuhkan heuristik menerima Heuristic sebagai parameter constructor sehingga mudah diganti.

```

// Manhattan.java
public int estimate(State state, Board board) {
    int dr = Math.abs(state.getRow() - board.getGoalRow());
    int dc = Math.abs(state.getCol() - board.getGoalCol());
    return dr + dc;
}

// CheckpointAware.java -- lewat checkpoint dulu
public int estimate(State state, Board board) {

```

```

    if (nextCheckp == total)
        return manhattan(state, goal);
    int[] cpPos = findCheckpoint(board, nextCheckp);
    return manhattan(state, cpPos) + manhattan(cpPos, goal);
}

```

IO

BoardParser.java membaca file .txt menggunakan BufferedReader dan try-with-resources, mem-parsing baris pertama sebagai N dan M, N baris berikutnya sebagai grid Tile[], dan N baris berikutnya sebagai cost[]. InputValidator dipanggil di beberapa titik untuk memastikan format valid sebelum diproses.

SolutionWriter.java menulis hasil solver ke file .txt berisi gerakan, total cost, iterasi, waktu, dan visualisasi papan tiap step di mana posisi karakter ditandai '@'.

UI

ConsoleUI.java mengurus seluruh interaksi dengan user di terminal, di mana ia meminta path file, memilih algoritma, memilih heuristik, menampilkan hasil, dan menawarkan playback serta simpan solusi. ConsoleUI tidak mengandung logika apapun, ia hanya mengkoordinasikan komponen lain.

Playback.java menyediakan navigasi interaktif di terminal menggunakan input keyboard, seperti 'd' untuk maju satu step, 'a' untuk mundur satu step, 'e' untuk lompat ke step tertentu, dan 'q' untuk keluar. method printStep() dibuat static sehingga bisa dipanggil dari ConsoleUI tanpa membuat objek Playback.

```

public static void printStep(SolverResult result, Board board, int
step) {
    // render papan dengan 'Z' di posisi karakter saat ini
    if (r == state.getRow() && c == state.getCol())
        System.out.print('Z');
    else
        System.out.print(board.getTile(r, c).getSymbol());
}

```

Bonus

Algoritma Tambahan 1 (BFS)

BFS adalah algoritma pencarian uninformed yang diimplementasikan sebagai bonus. Berbeda dari UCS/GBFS/A* yang menggunakan PriorityQueue, BFS

menggunakan Queue FIFO (ArrayDeque) sehingga semua state diperlakukan sama tanpa prioritas apapun. BFS menjamin solusi dengan jumlah gerakan paling sedikit namun tidak menjamin cost terkecil karena tidak mempertimbangkan bobot tiap tile. BFS tidak extend Solver karena struktur datanya berbeda, di mana visited ditandai saat add ke queue (bukan saat poll) karena tidak ada kemungkinan menemukan path lebih baik ke state yang sama di kemudian hari.

```
Queue<State> queue = new ArrayDeque<>();
if (!visited.contains(next)) {
    visited.add(next); // tandai saat add
    queue.add(next);
}
```

Algoritma Tambahan 2 (DFS)

DFS adalah algoritma pencarian uninformed kedua yang diimplementasikan sebagai bonus. DFS menggunakan Deque sebagai stack LIFO (Last In First Out) sehingga mengeksplorasi satu jalur sedalam mungkin sebelum backtrack dan mencoba jalur lain. DFS tidak menjamin solusi optimal dan bisa menghasilkan jalur yang sangat panjang. Seperti BFS, DFS tidak extend Solver dan visited ditandai saat push ke stack.

```
Deque<State> stack = new ArrayDeque<>();
stack.push(next); // masuk ke atas stack (LIFO)
State current = stack.pop(); // keluar dari atas stack
```

Perbandingan kelima algoritma:

Solver	Struktur Data	fCost
UCS	PriorityQueue	g
GBFS	PriorityQueue	h
A*	PriorityQueue	g + h
BFS	Queue (FIFO)	Tidak Ada
DFS	Stack (LIFO)	Tidak Ada

Heuristik Bonus 1 (CheckpointAware)

CheckpointAware adalah heuristik bonus pertama yang dirancang khusus untuk Ice Sliding Puzzle. Berbeda dari Manhattan biasa yang langsung mengestimasi jarak ke goal, CheckpointAware memperhitungkan bahwa karakter wajib melewati

checkpoint berikutnya sebelum bisa mencapai goal. Estimasinya adalah Manhattan dari posisi sekarang ke checkpoint berikutnya ditambah Manhattan dari checkpoint tersebut ke goal. Jika semua checkpoint sudah dilalui, estimasi langsung ke goal.

```
public int estimate(State state, Board board) {
    int nextCheckp = state.getNextCheckPoint();
    int total = board.getTotalCheckPoint();
    if (nextCheckp == total)
        return manhattan(state.getRow(), state.getCol(),
                           board.getGoalRow(), board.getGoalCol());
    int[] cpPos = findCheckpoint(board, nextCheckp);
    int toCheckp = manhattan(state.getRow(), state.getCol(), cpPos[0],
cpPos[1]);
    int toGoal = manhattan(cpPos[0], cpPos[1], board.getGoalRow(),
board.getGoalCol());
    return toCheckp + toGoal;
}
```

CheckpointAware tetap admissible karena menggunakan Manhattan distance (yang sudah terbukti admissible) untuk mengestimasi masing-masing bagian, dan rute nyata tidak mungkin lebih pendek dari estimasi Manhattan tersebut.

Heuristik Bonus 2 (Euclidean)

Euclidean adalah heuristik bonus kedua yang mengestimasi jarak menggunakan rumus jarak garis lurus (teorema Pythagoras). Euclidean selalu menghasilkan nilai yang lebih kecil atau sama dengan Manhattan karena jarak garis lurus tidak pernah melebihi jarak Manhattan pada grid 4 arah. Akibatnya Euclidean admissible namun kurang akurat dibanding Manhattan sehingga A* dengan Euclidean perlu mengeksplorasi lebih banyak node.

```
public int estimate(State state, Board board) {
    double dr = state.getRow() - board.getGoalRow();
    double dc = state.getCol() - board.getGoalCol();
    return (int) Math.sqrt((dr * dr) + (dc * dc));
}
```

Berikut merupakan perbandingan dari ketiga heuristik:

Heuristik	Formula	Admissible
Manhattan	$ dr + dc $	Ya
CheckpointAware	manhattan via checkpoint	Ya

Euclidean	$\sqrt{dr^2 + dc^2}$	Ya
-----------	----------------------	----

GUI

Program menyediakan antarmuka grafis berbasis JavaFX yang dapat dijalankan dengan menambahkan flag `--gui` saat menjalankan program. GUI terdiri dari tiga komponen utama yang masing-masing punya tanggung jawab berbeda.

GUIApp.java adalah entry point JavaFX yang extend Application dan mengatur layout utama menggunakan BorderPane: panel kiri berisi kontrol input (pilih file, algoritma, heuristik, tombol solve dan simpan), panel tengah berisi BoardPanel untuk rendering papan, dan panel bawah berisi ControlPanel untuk kontrol playback. Saat user menekan tombol Solve, GUIApp memanggil solver yang sesuai, menyimpan SolverResult, lalu mengoper hasilnya ke ControlPanel dan BoardPanel. Label infoLabel diperbarui secara realtime menampilkan gerakan, cost, iterasi, dan waktu eksekusi.

```
btnSolve.setOnAction(e -> {
    result = jalankanSolver(algo, board, heuristic);
    controlPanel.setResult(result, board);
    boardPanel.setBoard(board, result, 0);
    infoLabel.setText("Gerakan : " + result.getMoveAsString() + "\n"
+
                    "Cost      : " + result.getTotalCost() + "\n" +
...);
});
```

BoardPanel.java extend Pane JavaFX dan menggunakan Canvas dengan GraphicsContext untuk menggambar papan secara manual. Setiap tile digambar sebagai kotak berwarna sesuai tipenya (WALL hijau tua, PATH hitam kehijauan, LAVA merah, GOAL kuning emas, CHECKPOINT hijau, aktor kuning dengan label Z). Checkpoint yang sudah dilewati ditampilkan dengan label V sebagai feedback visual. Animasi sliding diimplementasikan menggunakan Timeline JavaFX dengan 12 keyframe per gerakan.

```
slideAnimation = new Timeline(new KeyFrame(Duration.millis(40), e -> {
    actorPixelRow += stepRow; // geser per frame
    actorPixelCol += stepCol;
    draw();                  // redraw seluruh canvas
}));
slideAnimation.setCycleCount(frames + 1);
```

```
slideAnimation.play();
```

ControlPanel.java extend VBox JavaFX dan menyediakan tiga tombol (prev, play/pause, next), label step saat ini, dan slider kecepatan 1–10. Auto-play diimplementasikan menggunakan Timeline kedua dengan interval yang dihitung dari nilai slider: $\text{delay} = 1000 - (\text{speed} \times 90)$ ms, sehingga kecepatan 1 menghasilkan delay 910ms dan kecepatan 10 menghasilkan delay 100ms. Saat slider diubah selama auto-play berjalan, Timeline dihentikan dan dibuat ulang dengan delay baru sehingga perubahan kecepatan langsung terasa.

```
speedSlider.valueProperty().addListener((obs, old, newVal) -> {  
    if (isPlaying) { stopAutoPlay(); startAutoPlay(); }  
});
```

Uji Coba

Test Case 1

Input:

77

XXXXXXX

X0****X

X**X**X

X****OX

X1***LX

XZ**X*X

XXXXXXX

999 999 999 999 999 999 999

999 3 5 2 8 1 999

999 7 4 999 6 9 999

999 2 8 3 5 4 999

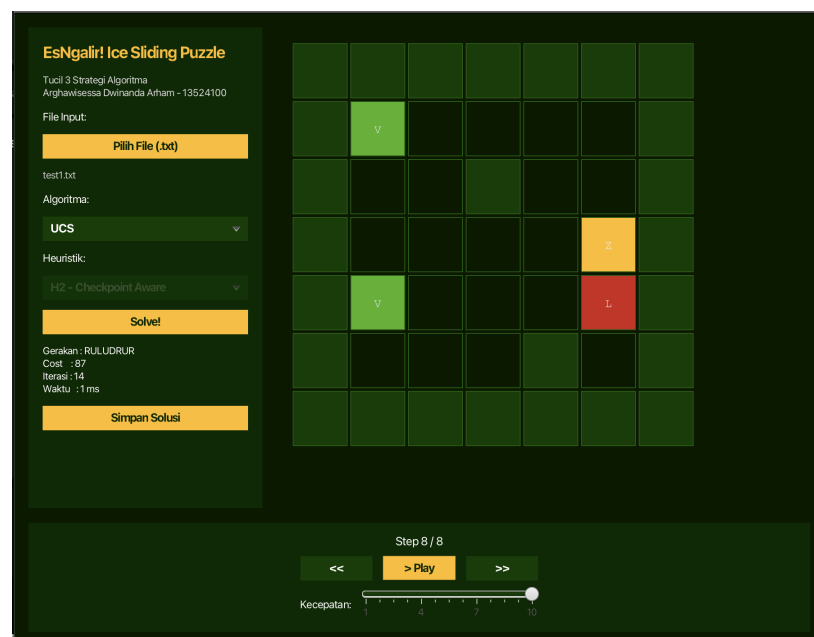
999 6 1 7 2 999 999

999 9 3 4 999 8 999

999 999 999 999 999 999 999

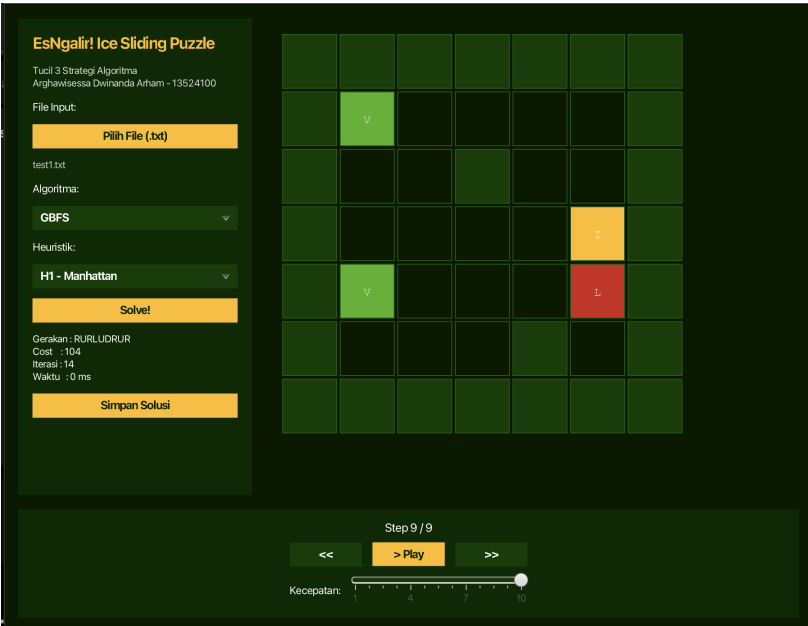
Output:

UCS:



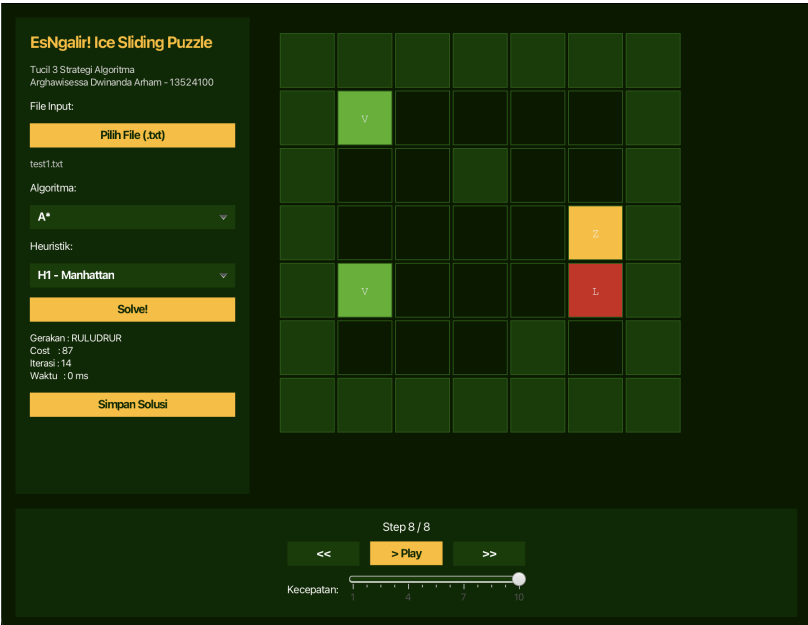
Gambar 3. Output Test Case 1 UCS

GBFS (H1):



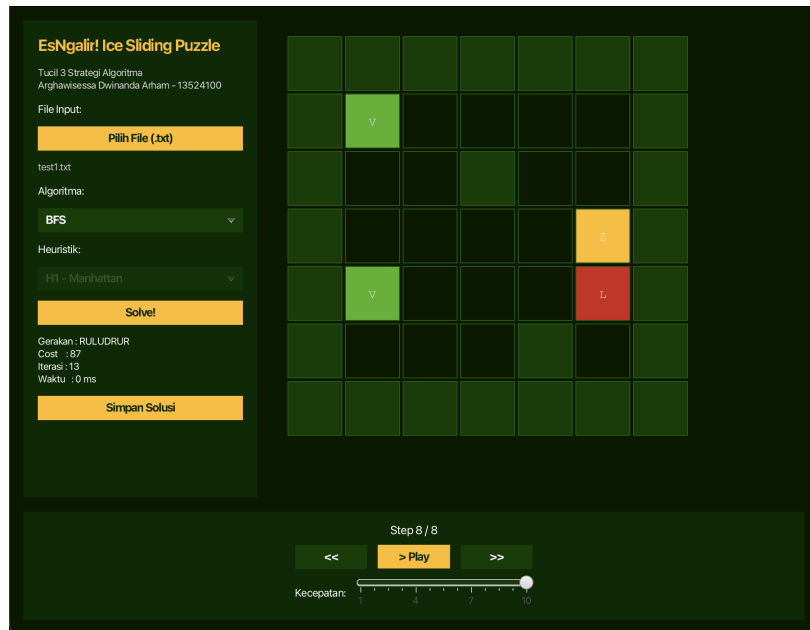
Gambar 4. Output Test Case 1 GBFS (H1)

A* (H1):



Gambar 5. Output Test Case 1 A* (H1)

BFS:



Gambar 6. Output Test Case 1 BFS

Test Case 2

Input:

5 5

XXXXX

XZ**X

X***X

X**OX

XXXXX

999 999 999 999 999

999 1 2 3 999

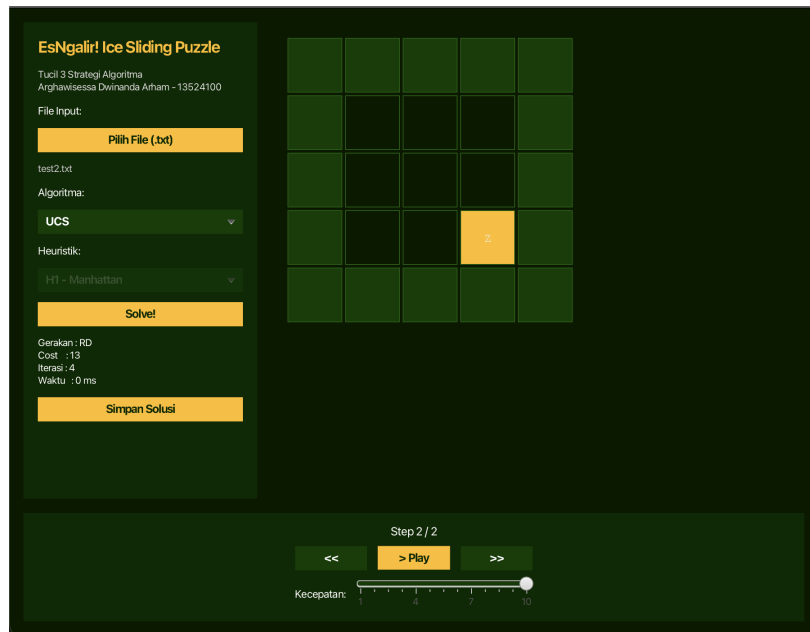
999 4 5 6 999

999 7 8 2 999

999 999 999 999 999

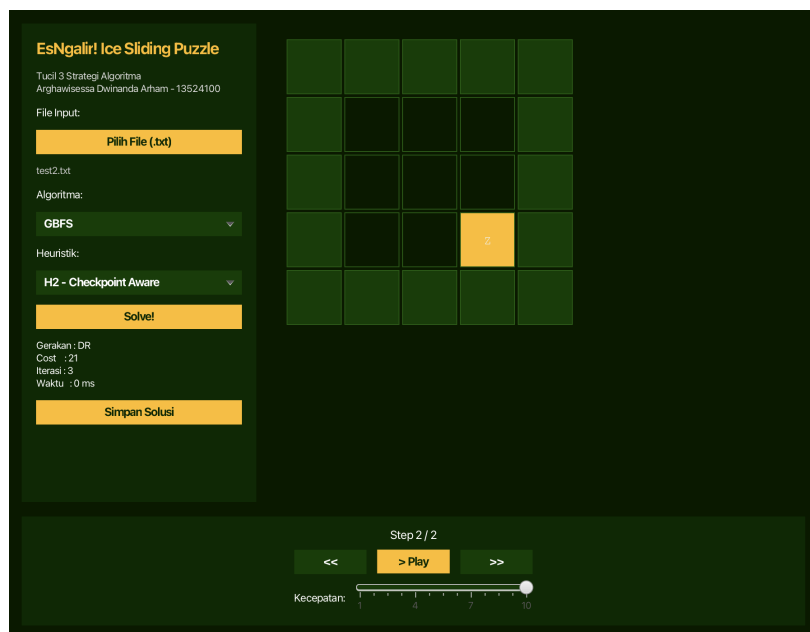
Output:

UCS:



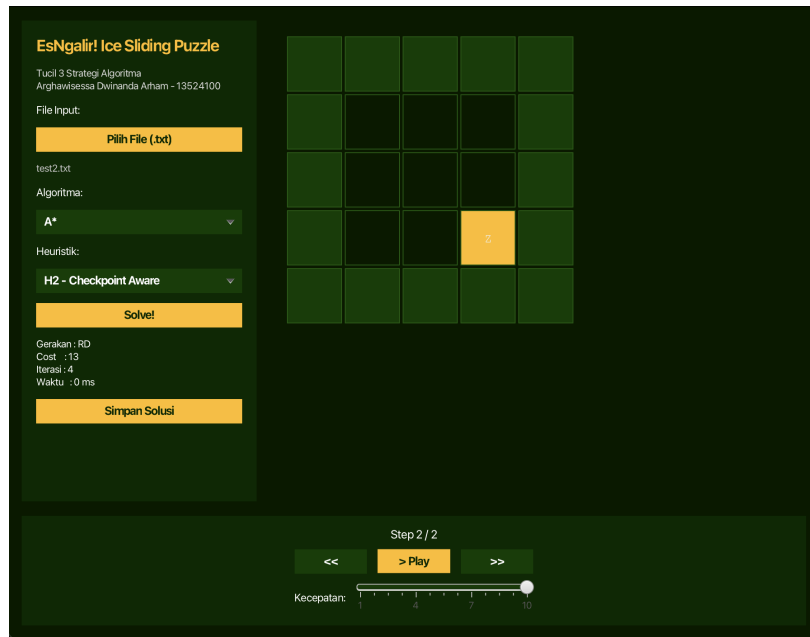
Gambar 7. Output Test Case 2 UCS

GBFS (H2):



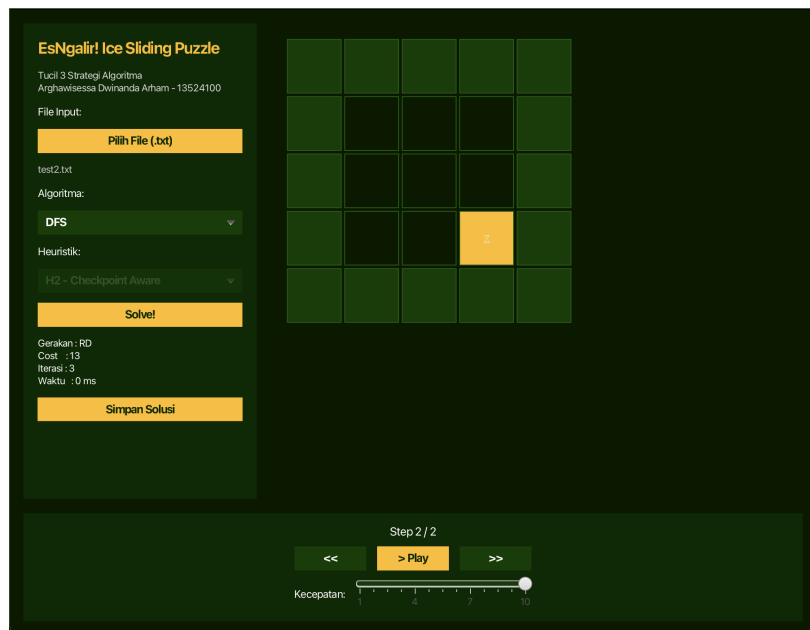
Gambar 8. Output Test Case 2 GBFS (H2)

A* (H2):



Gambar 9. Output Test Case 1 A* (H2)

DFS:



Gambar 10. Output Test Case 2 DFS

Test Case 3

Input:

57

XXXXXXX

XZ*L**X

X*****X

X***00X

XXXXXXX

999 999 999 999 999 999 999

999 1 2 999 3 4 999

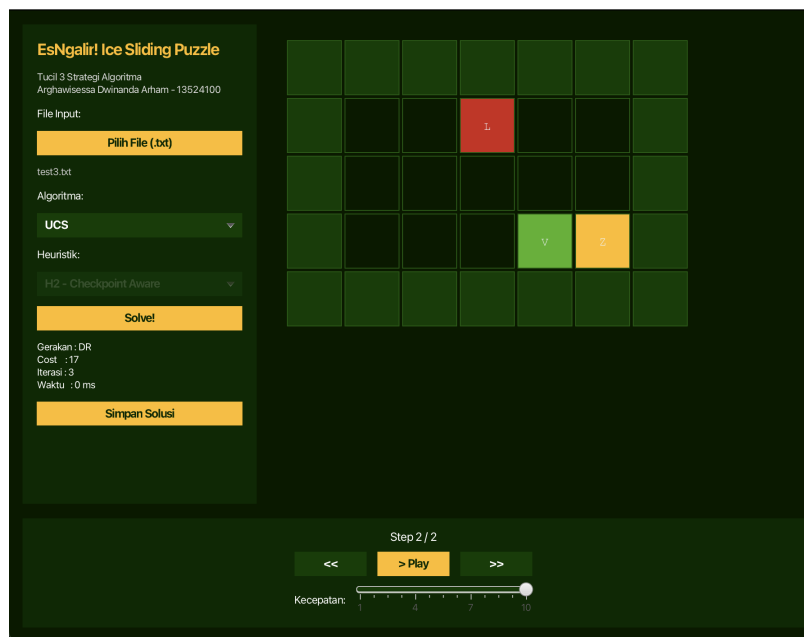
999 5 6 7 8 9 999

999 2 3 4 1 2 999

999 999 999 999 999 999 999

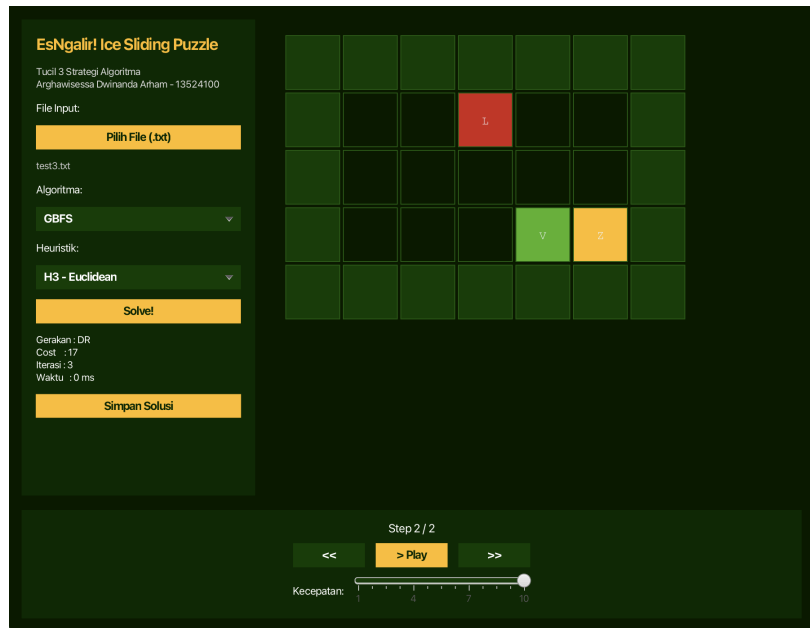
Output:

UCS:



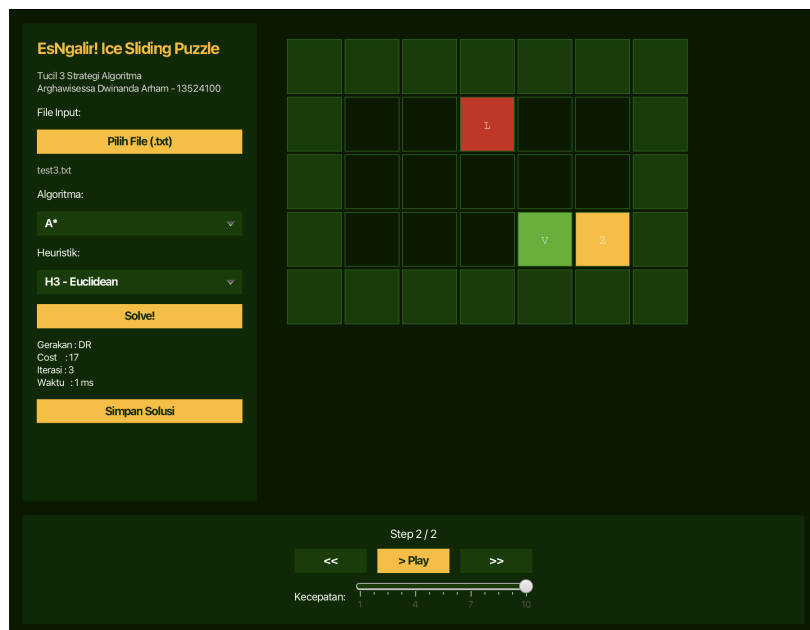
Gambar 11. Output Test Case 3 UCS

GBFS (H3):



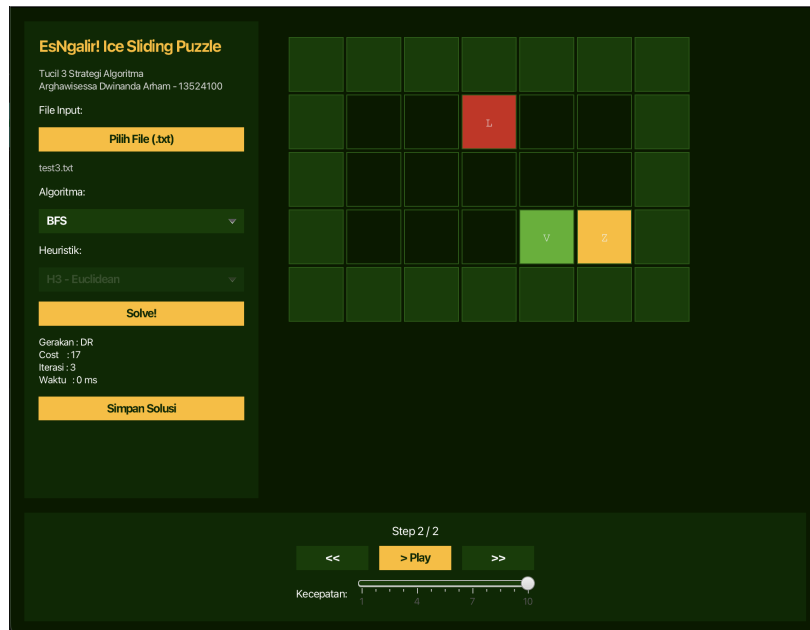
Gambar 12. Output Test Case 3 GBFS (H3)

A* (H3):



Gambar 13. Output Test Case 3 A* (H3)

BFS:



Gambar 14. Output Test Case 3 BFS

Test Case 4

Input:

5 5

XXXXX

XZ*XX

XXXXX

XX*OX

XXXXX

999 999 999 999 999

999 1 2 999 999

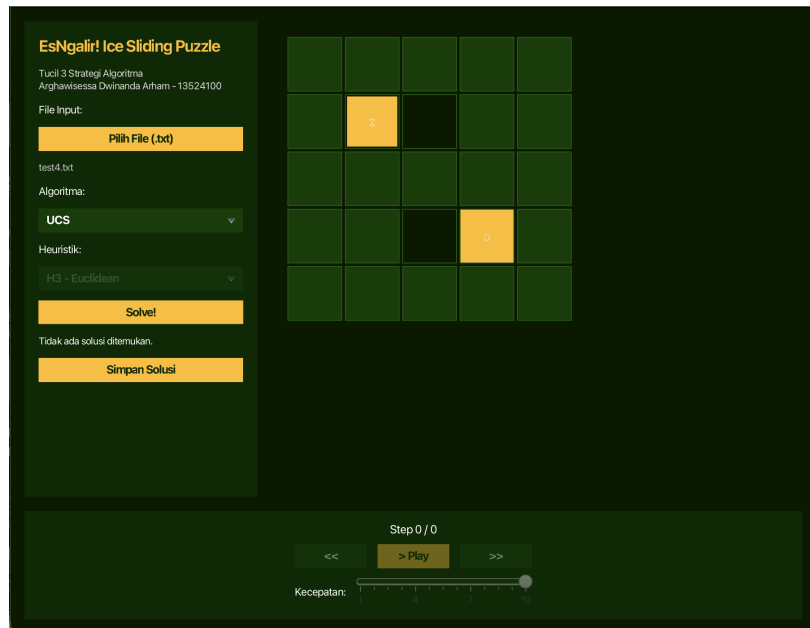
999 999 999 999 999

999 999 3 4 999

999 999 999 999 999

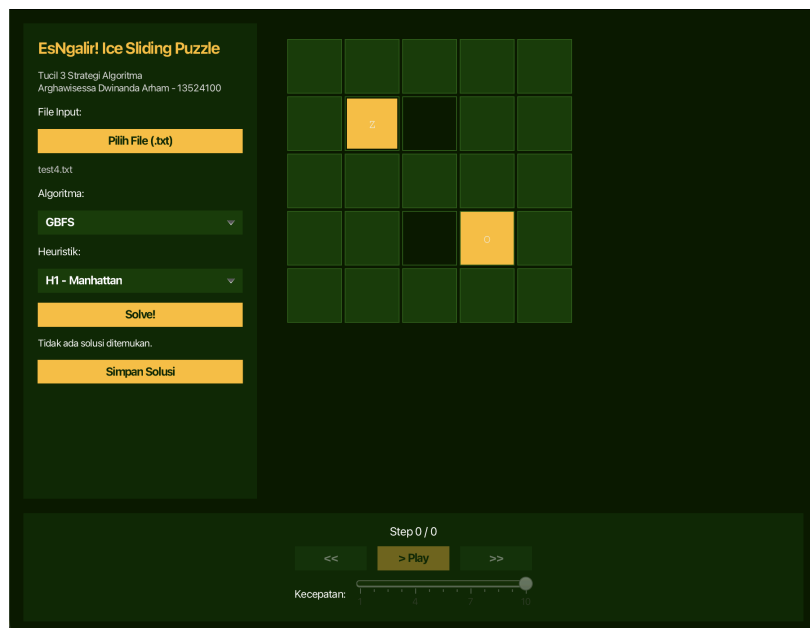
Output:

UCS:



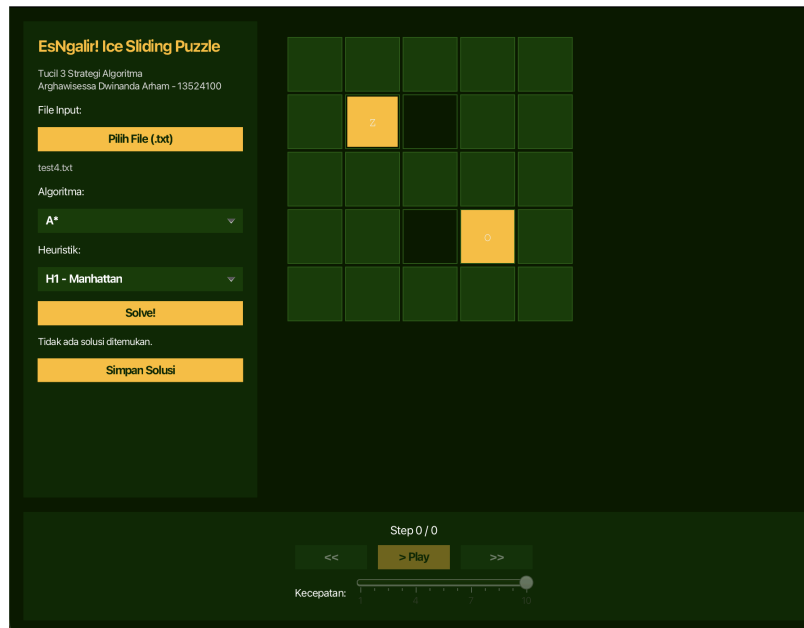
Gambar 15. Output Test Case 4 UCS

GBFS (H1):



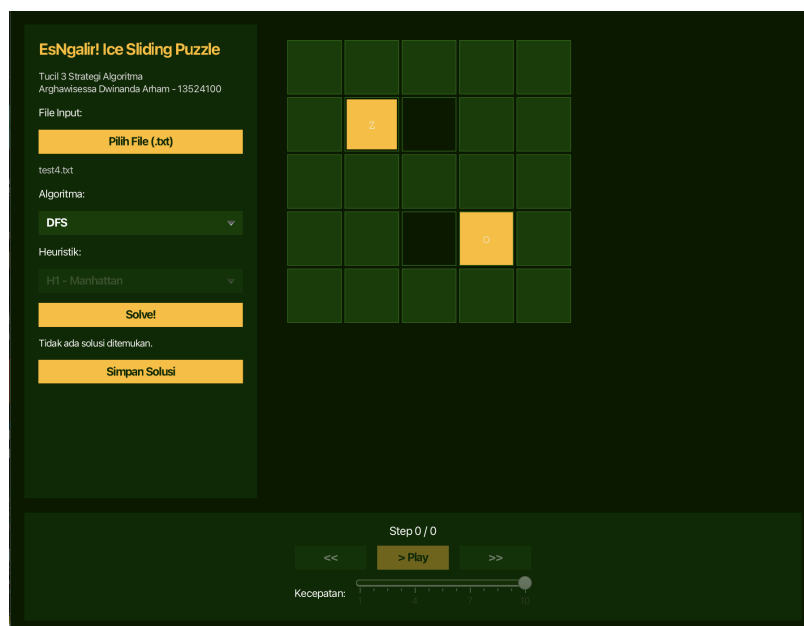
Gambar 16. Output Test Case 4 GBFS (H1)

A* (H1):



Gambar 17. Output Test Case 4 A* (H1)

DFS:



Gambar 18. Output Test Case 4 DFS

Test Case 5

Input:

77

XXXXXXX

XO***1X

X*XXX*X

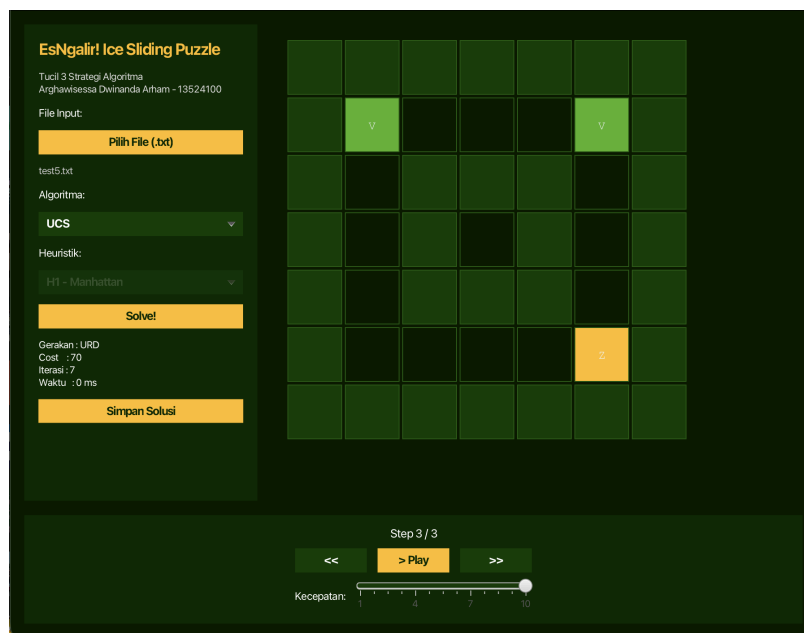
```

X*X*X*X
X*XXX*X
XZ***OX
XXXXXXX
999 999 999 999 999 999 999
999 1 2 3 4 5 999
999 6 999 999 999 7 999
999 8 999 9 999 10 999
999 11 999 999 999 12 999
999 2 3 4 5 1 999
999 999 999 999 999 999 999

```

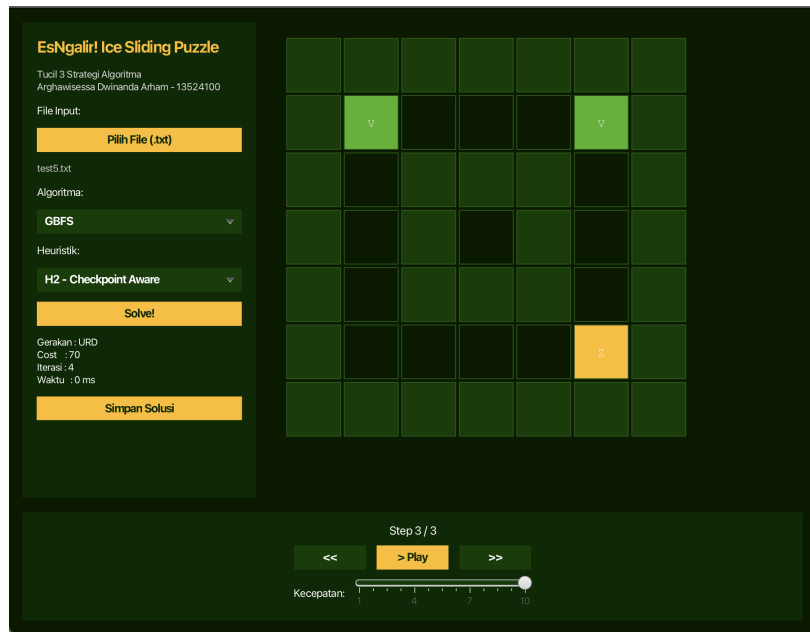
Output:

UCS:



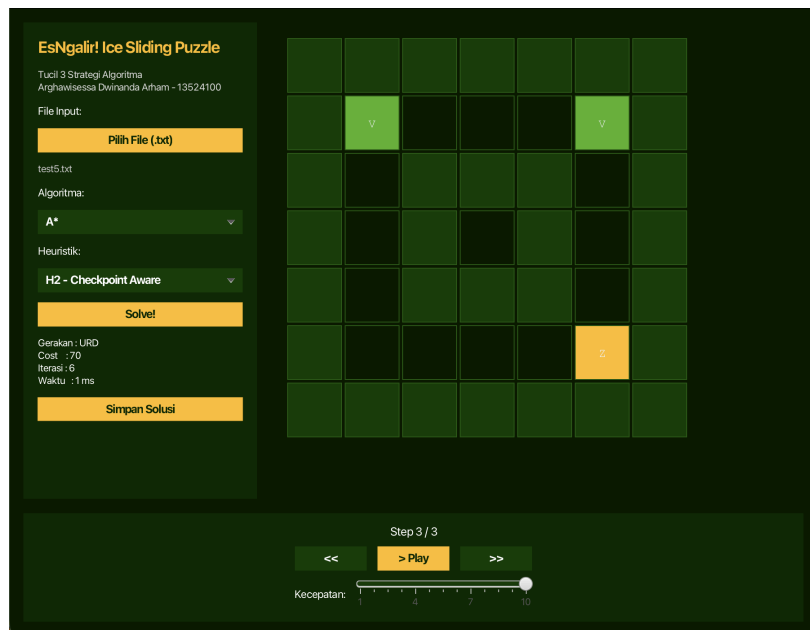
Gambar 19. Output Test Case 5 UCS

GBFS (H2):



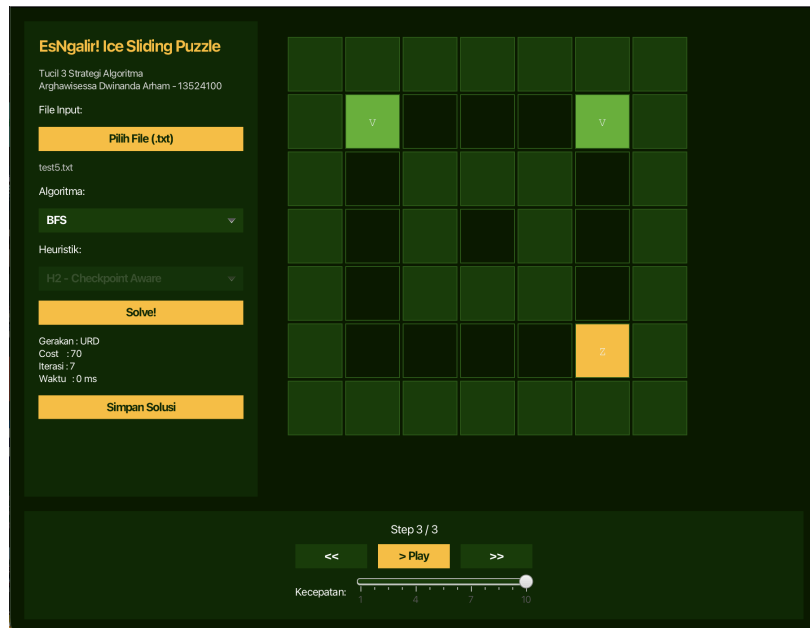
Gambar 20. Output Test Case 5 GBFS (H2)

A* (H2):



Gambar 21. Output Test Case 5 A* (H2)

BFS:



Gambar 22. Output Test Case 5 DFS

Analisis dan Kesimpulan

Analisis

Rekapitulasi Hasil Uji Coba

Test Case 1:

Algoritma	Gerakan	Cost	Iterasi
UCS	RULUDRUR	87	13
GBFS (H1)	RURLUDRUR	104	14
A* (H1)	RULUDRUR	87	14
BFS	RULUDRUR	87	13

Test Case 2:

Algoritma	Gerakan	Cost	Iterasi
UCS	RD	13	4
GBFS (H2)	DR	21	3
A* (H2)	RD	13	4
DFS	RD	13	3

Test Case 3:

Algoritma	Gerakan	Cost	Iterasi
UCS	DR	17	3
GBFS (H3)	DR	17	3
A* (H3)	DR	17	3
BFS	DR	17	3

Test Case 4:

Algoritma	Gerakan	Cost	Iterasi
UCS	-	-	-
GBFS (H1)	-	-	-
A* (H1)	-	-	-

DFS	-	-	-
-----	---	---	---

Test Case 5:

Algoritma	Gerakan	Cost	Iterasi
UCS	URD	70	7
GBFS (H2)	URD	70	4
A* (H2)	URD	70	6
BFS	URD	70	7

Analisis per Algoritma

UCS (Uniform Cost Search):

UCS konsisten menemukan solusi optimal di semua test case yang memiliki solusi, yaitu cost 87 pada test case 1, cost 13 pada test case 2, cost 17 pada test case 3, dan cost 70 pada test case 5. Hal ini sesuai dengan sifat teoretis UCS yang menjamin optimalitas selama semua cost bernilai non-negatif. Jumlah iterasi UCS relatif moderat (13, 4, 3, 7) karena UCS tidak memiliki heuristik sehingga mengeksplorasi node berdasarkan cost nyata saja tanpa perkiraan arah ke goal.

GBFS (Greedy Best First Search):

GBFS menunjukkan perilaku yang tidak konsisten dalam hal optimalitas, sesuai teori. Pada test case 1 dengan H1 (Manhattan), GBFS menemukan jalur RURLUDRUR dengan cost 104 yang tidak optimal dibanding solusi optimal 87. Pada test case 2 dengan H2 (CheckpointAware), GBFS menemukan jalur DR dengan cost 21 yang juga tidak optimal dibanding RD dengan cost 13. Namun pada test case 3 dan 5, GBFS berhasil menemukan solusi optimal karena hanya ada satu jalur valid. Keunggulan GBFS terlihat pada test case 5 di mana GBFS hanya butuh 4 iterasi, jauh lebih sedikit dibanding UCS (7 iterasi) dan A* (6 iterasi), membuktikan bahwa GBFS memang paling cepat meskipun tidak selalu optimal.

A* (A-Star):

A* berhasil menemukan solusi optimal di semua test case yang memiliki solusi: cost 87, 13, 17, dan 70 secara berurutan. Dengan heuristik H1 (Manhattan) pada test case 1, A* membutuhkan 14 iterasi, lebih banyak dari UCS (13 iterasi). Ini menunjukkan bahwa Manhattan distance kurang akurat untuk test case dengan checkpoint

karena tidak memperhitungkan kewajiban melewati checkpoint. Sebaliknya pada test case 5 dengan H2 (CheckpointAware), A* membutuhkan 6 iterasi dibanding UCS yang membutuhkan 7 iterasi, membuktikan bahwa heuristik yang lebih akurat menghasilkan eksplorasi yang lebih efisien.

BFS (Breadth First Search (Bonus)):

BFS berhasil menemukan solusi optimal pada test case 1, 3, dan 5 dengan cost yang sama dengan UCS dan A*. Namun secara teori BFS tidak menjamin cost optimal, hanya menjamin jumlah gerakan (step) paling sedikit. Kebetulan pada test case ini jalur dengan gerakan paling sedikit juga merupakan jalur dengan cost terendah. Jumlah iterasi BFS sama dengan UCS pada beberapa test case (13 pada test case 1, 7 pada test case 5) karena keduanya mengeksplorasi secara menyeluruh tanpa heuristik.

DFS (Depth First Search (Bonus)):

DFS memberikan hasil yang menarik: pada test case 2, DFS berhasil menemukan solusi RD dengan cost 13 (optimal) hanya dengan 3 iterasi, lebih sedikit dari UCS (4 iterasi) dan A* (4 iterasi). Ini terjadi secara kebetulan karena urutan eksplorasi DFS (RIGHT dieksplorasi sebelum DOWN berdasarkan urutan `Direction.values()`) langsung mengarah ke jalur yang optimal. Namun DFS tidak menjamin optimalitas secara umum. Pada test case 4, DFS juga berhasil mendeteksi ketiadaan solusi dengan benar. Kelemahan DFS adalah tidak dapat diprediksi dan sangat bergantung pada urutan eksplorasi arah.

Analisis Heuristik

Pada test case 1, A* dengan H1 (Manhattan) membutuhkan 14 iterasi sedangkan UCS hanya 13 iterasi. Ini menunjukkan bahwa Manhattan distance sebagai heuristik untuk test case dengan checkpoint justru kurang efisien dibanding tanpa heuristik sama sekali. Hal ini terjadi karena Manhattan mengabaikan keberadaan checkpoint sehingga estimasinya tidak mencerminkan kondisi nyata, karakter harus memutar melewati checkpoint yang mungkin berlawanan arah dengan goal.

Pada test case 5, A* dengan H2 (CheckpointAware) membutuhkan 6 iterasi dibanding UCS yang membutuhkan 7 iterasi. CheckpointAware memberikan estimasi yang lebih realistis dengan memperhitungkan kewajiban melewati

checkpoint, sehingga A* dapat memangkas cabang yang tidak menjanjikan lebih efektif.

Pada test case 3, A* dengan H3 (Euclidean) membutuhkan 3 iterasi sama dengan UCS, GBFS, dan BFS. Ini terjadi karena test case 3 memiliki ruang pencarian yang sangat kecil (hanya satu jalur valid) sehingga perbedaan heuristik tidak berpengaruh.

Maka kesimpulannya adalah CheckpointAware (H2) terbukti menjadi heuristik terbaik untuk Ice Sliding Puzzle karena memperhitungkan aturan inti permainan. Manhattan (H1) cocok digunakan hanya untuk papan tanpa checkpoint. Euclidean (H3) secara konsisten menghasilkan estimasi paling rendah sehingga paling tidak efisien di antara ketiganya.

Kesimpulan

Program EsNgalir berhasil mengimplementasikan lima algoritma pathfinding (UCS, GBFS, A*, BFS, DFS) beserta tiga heuristik (Manhattan, CheckpointAware, Euclidean) untuk menyelesaikan Ice Sliding Puzzle. Berdasarkan hasil uji coba pada lima test case dengan karakteristik berbeda, dapat ditarik kesimpulan sebagai berikut.

Pertama, algoritma yang menjamin solusi optimal adalah UCS, A* (dengan heuristik admissible), dan BFS (dalam jumlah gerakan). Dari ketiganya, A* dengan heuristik CheckpointAware memberikan keseimbangan terbaik antara kecepatan dan optimalitas karena dapat memangkas ruang pencarian secara lebih efektif dibanding UCS.

Kedua, GBFS adalah algoritma paling cepat dalam hal jumlah iterasi namun tidak menjamin optimalitas. Pada papan dengan satu-satunya jalur valid, GBFS secara kebetulan menemukan solusi optimal. Namun pada papan dengan banyak pilihan jalur, GBFS cenderung menghasilkan solusi suboptimal.

Ketiga, DFS menunjukkan perilaku yang tidak dapat diprediksi: kadang menemukan solusi optimal (test case 2 dan 3) dan kadang tidak (tergantung urutan eksplorasi). DFS tidak direkomendasikan untuk kasus yang membutuhkan solusi optimal.

Keempat, pemilihan heuristik sangat menentukan efisiensi A*. CheckpointAware terbukti superior dibanding Manhattan dan Euclidean untuk Ice Sliding Puzzle karena dirancang khusus dengan mempertimbangkan aturan checkpoint permainan.

Manhattan masih layak digunakan untuk papan sederhana tanpa checkpoint, sedangkan Euclidean paling tidak efisien karena underestimate-nya terlalu besar.

Kelima, program berhasil menangani seluruh kasus tepi dengan benar: lava menyebabkan game over, checkpoint yang dilewati salah urutan menyebabkan game over, keluar papan menyebabkan game over, dan ketiadaan solusi terdeteksi dengan benar oleh semua algoritma.

Lampiran

Tabel Checklist:

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	V	
2	Program berhasil dijalankan	V	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	V	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	V	
5	Program memiliki Graphical User Interface (GUI)	V	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma BFS dan Algoritma DFS)	V	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik CheckpointAware dan Heuristik Euclidean)	V	

Pernyataan:

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Arghawisesa Dwinanda Arham

Link GitHub: https://github.com/arghaarham/Tucil3_13524100

Link Figma: <https://s.id/FigmaTucil3StimaArgha>

Referensi

- [1] Nur Ulfa Maulidevi, "Penentuan Rute (Route/Path Planning) Bagian 1: BFS, DFS, UCS, Greedy Best First Search", Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika STEI ITB, 2026.
- [2] Nur Ulfa Maulidevi & Rinaldi M, "Penentuan Rute (Route/Path Planning) Bagian 2: Algoritma A*", Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika STEI ITB, 2026.
- [3] "Spesifikasi Tugas Kecil 3 IF2211 Strategi Algoritma: Ice Sliding Puzzle Solver", Semester II Tahun 2025/2026, Institut Teknologi Bandung, 2026.
- [4] GeeksforGeeks, "Manhattan Distance", Diakses Mei 2026.
<https://www.geeksforgeeks.org/data-science/manhattan-distance/>
- [5] GeeksforGeeks, "A* Search Algorithm", Diakses Mei 2026.
<https://www.geeksforgeeks.org/dsa/a-search-algorithm/>
- [6] GeeksforGeeks, "Greedy Best First Search in AI", Diakses Mei 2026.
<https://www.geeksforgeeks.org/artificial-intelligence/greedy-best-first-search-in-ai/>
- [7] Smit Ahire, "How Computers Solve the Slider Puzzle", Medium, November 2025.
<https://medium.com/@smit.ahire23/how-computers-solve-the-slider-puzzle-49e82a026ea8>